# Spring Data Day3

---

## Spring Data Access

JDBC · Spring JDBC · ORM · Hibernate · JPA

---

### Course Outline

1. Using JDBC in the Spring Framework
2. Spring JDBC Module
3. Spring ORM
4. Spring ORM using Hibernate
5. Spring ORM Hibernate Integration
6. Spring ORM using JPA
7. Spring ORM JPA Integration

> Progression: raw JDBC → JdbcTemplate → ORM concepts → Hibernate → JPA

---

## Lesson 1

### Using JDBC in the Spring Framework

---

### What is JDBC?

- **J**ava **D**atabase **C**onnectivity: the standard API ( `java.sql` ) for relational databases
- Key types: `DriverManager` , `DataSource` , `Connection` , `Statement` , `PreparedStatement` , `ResultSet`
- Vendor-specific drivers implement the API (MySQL, PostgreSQL, Oracle, H2…)

### Typical steps

1. Obtain a `Connection`
2. Create a `PreparedStatement`
3. Execute the SQL
4. Process the `ResultSet`
5. Handle `SQLException`
6. Close `ResultSet` , `Statement` , `Connection`

---

### Plain JDBC Example

```
public Employee findById(int id) {
    String sql = "SELECT id, name, salary FROM employee WHERE id = ?";
    try (Connection con = DriverManager.getConnection(URL, USER, PASS);
         PreparedStatement ps = con.prepareStatement(sql)) {

        ps.setInt(1, id);
        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                return new Employee(rs.getInt("id"),
                                    rs.getString("name"),
                                    rs.getDouble("salary"));
            }
        }
    } catch (SQLException e) {
        throw new RuntimeException(e);   // checked exception noise
    }
    return null;
}
```

---

### Problems with Plain JDBC

| Problem | Consequence |
|---|---|
| Repetitive boilerplate | Same 15 lines in every DAO method |
| Manual resource handling | Connection leaks, exhausted pools |
| Checked `SQLException` | Cluttered code, vendor-specific error codes |
| Manual transaction code | `setAutoCommit(false)`, `commit`, `rollback` everywhere |
| Hard-coded connection info | Difficult to test and configure |

**Spring's answer:** inversion of control, template classes, a consistent unchecked exception hierarchy, declarative transactions.

---

### Spring Manages the DataSource

Never open connections by hand. Define a `DataSource` bean and inject it.

```
@Configuration
public class DataSourceConfig {

    @Bean
    public DataSource dataSource() {
        HikariConfig cfg = new HikariConfig();
        cfg.setJdbcUrl("jdbc:postgresql://localhost:5432/company");
        cfg.setUsername("app");
```

```
        cfg.setPassword("secret");
        cfg.setMaximumPoolSize(10);
        return new HikariDataSource(cfg);
    }
}
```

- `DriverManagerDataSource` : simple, **no pooling**, for tests only
- `HikariCP` : production connection pool (Spring Boot default)
- `EmbeddedDatabaseBuilder` : in-memory H2/HSQL for tests

---

## Embedded DB for Development

```
@Bean
public DataSource dataSource() {
    return new EmbeddedDatabaseBuilder()
            .setType(EmbeddedDatabaseType.H2)
            .addScript("classpath:schema.sql")
            .addScript("classpath:data.sql")
            .build();
}
```

`schema.sql`

```
CREATE TABLE employee (
    id     INT PRIMARY KEY AUTO_INCREMENT,
    name   VARCHAR(100) NOT NULL,
    salary DOUBLE
);
```

---

## Using JDBC with Spring: DataSourceUtils

Spring-managed connections take part in Spring transactions.

```
@Repository
public class LegacyEmployeeDao {
    private final DataSource ds;
    public LegacyEmployeeDao(DataSource ds) { this.ds = ds; }

    public int count() throws SQLException {
        Connection con = DataSourceUtils.getConnection(ds); // tx-aware
        try (PreparedStatement ps =
                 con.prepareStatement("SELECT COUNT(*) FROM employee");
             ResultSet rs = ps.executeQuery()) {
            rs.next();
            return rs.getInt(1);
        } finally {
```

```
            DataSourceUtils.releaseConnection(con, ds);
        }
    }
}
```

This is still verbose, which is why Lesson 2 introduces `JdbcTemplate`.

---

### Lab 1: JDBC in Spring

1. Create a Maven project with `spring-context`, `spring-jdbc`, `h2`, `HikariCP`.
2. Write `schema.sql` creating tables `department(id, name)` and `employee(id, name, salary, department_id)`, plus `data.sql` with 3 departments and 6 employees.
3. Create `DataSourceConfig` with a **HikariCP** `DataSource` bean (pool size 5).
4. Create `EmbeddedDatabaseBuilder` version of the same bean and switch between them using `@Profile("dev")` and `@Profile("prod")`.
5. Write `PlainEmployeeDao` using **plain JDBC** (`DataSourceUtils.getConnection`, `PreparedStatement`, `ResultSet`) with methods `findById`, `findAll`, `count`.
6. In `Main`, load the context, print all employees, and print the pool class name (`ds.getClass()`).

---

## Lesson 2

### Spring JDBC Module

---

### Spring JDBC: What You Get

- **`JdbcTemplate`**: executes SQL, handles connections, statements, exceptions
- **`NamedParameterJdbcTemplate`**: `:name` parameters instead of `?`
- **`SimpleJdbcInsert`** / **`SimpleJdbcCall`**: metadata-driven inserts and stored procedures
- **`RowMapper`** / **`ResultSetExtractor`** / **`RowCallbackHandler`**: result mapping
- **`DataAccessException`** **hierarchy**: unchecked, vendor-neutral
- **`JdbcTransactionManager`** (`DataSourceTransactionManager`): transactions

Spring does: open connection, prepare, execute, close, translate exceptions. You do: SQL, parameters, row mapping.

---

### Configuring JdbcTemplate

```
@Configuration
@ComponentScan("com.example")
public class AppConfig {
```

```
    @Bean
    public DataSource dataSource() { /* Lesson 1 */ }

    @Bean
    public JdbcTemplate jdbcTemplate(DataSource ds) {
        return new JdbcTemplate(ds);
    }

    @Bean
    public NamedParameterJdbcTemplate namedTemplate(DataSource ds) {
        return new NamedParameterJdbcTemplate(ds);
    }
}
```

---

## The Domain Class

```
public class Employee {
    private Integer id;
    private String name;
    private double salary;

    public Employee() {}
    public Employee(Integer id, String name, double salary) {
        this.id = id; this.name = name; this.salary = salary;
    }
    // getters and setters omitted
}
```

---

## Querying with JdbcTemplate

```
@Repository
public class EmployeeDao {
    private final JdbcTemplate jdbc;

    public EmployeeDao(JdbcTemplate jdbc) { this.jdbc = jdbc; }

    // single value
    public int count() {
        return jdbc.queryForObject("SELECT COUNT(*) FROM employee", Integer.class);
    }

    // single row with a lambda RowMapper
    public Employee findById(int id) {
        return jdbc.queryForObject(
            "SELECT id, name, salary FROM employee WHERE id = ?",
            (rs, rowNum) -> new Employee(rs.getInt("id"),
                                         rs.getString("name"),
                                         rs.getDouble("salary")),
```

```
            id);
    }

    // list of rows
    public List<Employee> findAll() {
        return jdbc.query("SELECT * FROM employee",
                new BeanPropertyRowMapper<>(Employee.class));
    }
}
```

---

### Insert, Update, Delete

```
public int insert(Employee e) {
    return jdbc.update(
        "INSERT INTO employee (name, salary) VALUES (?, ?)",
        e.getName(), e.getSalary());
}

public int updateSalary(int id, double salary) {
    return jdbc.update(
        "UPDATE employee SET salary = ? WHERE id = ?", salary, id);
}

public int delete(int id) {
    return jdbc.update("DELETE FROM employee WHERE id = ?", id);
}
```

`update()` returns the number of affected rows.

---

### Getting the Generated Key

```
public int insertAndReturnId(Employee e) {
    KeyHolder keys = new GeneratedKeyHolder();
    jdbc.update(con -> {
        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO employee (name, salary) VALUES (?, ?)",
            new String[] {"id"});
        ps.setString(1, e.getName());
        ps.setDouble(2, e.getSalary());
        return ps;
    }, keys);
    return keys.getKey().intValue();
}
```

---

### Named Parameters

```
public List<Employee> findBySalaryRange(double min, double max) {
    String sql = """
        SELECT * FROM employee
        WHERE salary BETWEEN :min AND :max
        ORDER BY salary DESC
        """;
    Map<String, Object> params = Map.of("min", min, "max", max);
    return named.query(sql, params,
            new BeanPropertyRowMapper<>(Employee.class));
}

public int save(Employee e) {
    return named.update(
        "INSERT INTO employee (name, salary) VALUES (:name, :salary)",
        new BeanPropertySqlParameterSource(e));
}
```

More readable and safer than positional `?`.

---

### Exception Translation

`SQLException` (checked, vendor codes) becomes `DataAccessException` (unchecked).

```
DataAccessException
├── DuplicateKeyException
├── DataIntegrityViolationException
├── EmptyResultDataAccessException
├── IncorrectResultSizeDataAccessException
├── CannotGetJdbcConnectionException
├── BadSqlGrammarException
└── DeadlockLoserDataAccessException
```

```
try {
    dao.insert(emp);
} catch (DuplicateKeyException ex) {
    // portable: same code for MySQL, Oracle, PostgreSQL
}
```

---

### Transactions with Spring JDBC

```
@Configuration
@EnableTransactionManagement
public class TxConfig {
    @Bean
    public PlatformTransactionManager txManager(DataSource ds) {
        return new DataSourceTransactionManager(ds);
```

```
    }
}
```

```
@Service
public class PayrollService {
    private final EmployeeDao dao;
    public PayrollService(EmployeeDao dao) { this.dao = dao; }

    @Transactional
    public void transferBonus(int fromId, int toId, double amount) {
        dao.updateSalary(fromId, dao.findById(fromId).getSalary() - amount);
        dao.updateSalary(toId,   dao.findById(toId).getSalary() + amount);
        // RuntimeException here => both updates roll back
    }
}
```

---

### Running the Application

```
public class Main {
    public static void main(String[] args) {
        try (var ctx = new AnnotationConfigApplicationContext(AppConfig.class)) {
            EmployeeDao dao = ctx.getBean(EmployeeDao.class);
            dao.insert(new Employee(null, "Ali", 4500));
            dao.findAll().forEach(e ->
                System.out.println(e.getName() + " earns " + e.getSalary()));
        }
    }
}
```

### Maven dependency

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-jdbc</artifactId>
  <version>6.1.x</version>
</dependency>
```

---

## Lab 2: Spring JDBC Module

### Tasks

1. Create the `Employee` class and `EmployeeRowMapper` (implement `RowMapper<Employee>`).
2. Create `EmployeeDao` (`@Repository`) with:
   - `insert(Employee)` returning the **generated id** (`KeyHolder`)
   - `findById`, `findAll`, `update`, `delete`
   - `findBySalaryRange(min, max)` using `NamedParameterJdbcTemplate`

- `countByDepartment()` returning `Map<String, Integer>` (use `ResultSetExtractor`)

3. Add `insertBatch(List<Employee>)` using `batchUpdate` and insert 100 employees.
4. Create `PayrollService` with `@Transactional transferBonus(fromId, toId, amount)`, which updates two salaries.
5. Force a `RuntimeException` after the first update and prove the **rollback** by reading salaries afterward.
6. Catch `DuplicateKeyException` when inserting an existing id, and print a friendly message.

---

## Lesson 3

## Spring ORM

---

### Why ORM?

**Object-Relational Mapping** maps Java objects to relational tables.

| Object world | Relational world |
|---|---|
| Class | Table |
| Field | Column |
| Object instance | Row |
| Reference / collection | Foreign key / join |
| Inheritance | (no direct equivalent) |

The **impedance mismatch**: identity, inheritance, associations, and navigation differ between the two models. An ORM bridges the gap so you work with objects instead of SQL and `ResultSet`s.

---

### What Spring ORM Provides

Spring does **not** replace Hibernate/JPA. It integrates them.

- Consistent **configuration** of `SessionFactory` / `EntityManagerFactory` as Spring beans
- **Transaction management** (`HibernateTransactionManager`, `JpaTransactionManager`)
- **Exception translation** to `DataAccessException` via `@Repository`
- **Resource management**: sessions/entity managers tied to the transaction
- Easy **testing** (swap the DataSource, use embedded DBs)

Supported: Hibernate, JPA, (legacy: JDO, iBATIS)

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-orm</artifactId>
</dependency>
```

---

## Key Spring ORM Building Blocks

| Class / Annotation | Role |
|---|---|
| `LocalSessionFactoryBean` | Builds a Hibernate `SessionFactory` |
| `LocalContainerEntityManagerFactoryBean` | Builds a JPA `EntityManagerFactory` |
| `HibernateTransactionManager` | Transactions for native Hibernate |
| `JpaTransactionManager` | Transactions for JPA |
| `@Repository` | Marks a DAO and enables exception translation |
| `PersistenceExceptionTranslationPostProcessor` | Applies translation to `@Repository` beans |
| `@Transactional` | Declarative transaction boundary |

---

## Exception Translation in ORM

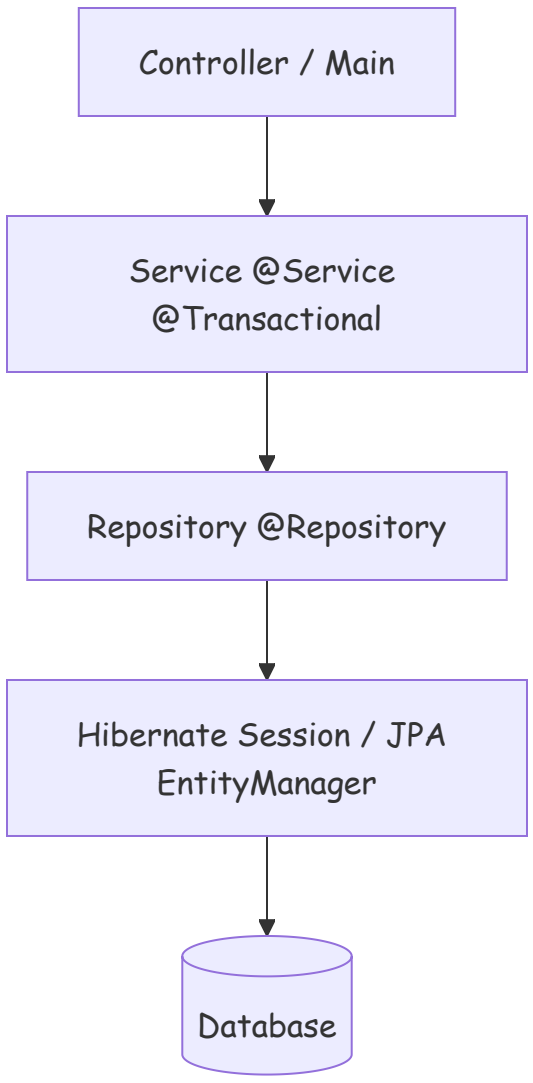
```
@Bean
public PersistenceExceptionTranslationPostProcessor exceptionTranslation() {
    return new PersistenceExceptionTranslationPostProcessor();
}
```

```
@Repository   // proxy translates HibernateException/PersistenceException
public class EmployeeRepository { ... }
```

Result: service code catches `DataAccessException` subclasses regardless of the persistence technology, so swapping Hibernate for JPA does not touch the service layer.

---

## Layered Architecture




- **Service** defines transaction boundaries
- **Repository** contains persistence logic only
- **Entities** are plain annotated classes

---

## Transaction Manager Abstraction

| Technology | Implementation |
| --- | --- |
| JDBC | `DataSourceTransactionManager` |
| Hibernate | `HibernateTransactionManager` |
| JPA | `JpaTransactionManager` |

The code uses `@Transactional`; only the bean definition changes.

---

## @Transactional Attributes

```
@Transactional(
    propagation = Propagation.REQUIRED,     // join or create
    isolation   = Isolation.READ_COMMITTED,
    readOnly    = false,
    timeout     = 30,
    rollbackFor = Exception.class           // default: RuntimeException only
)
public void hire(Employee e) { ... }
```

Common propagation values: `REQUIRED` (default), `REQUIRES_NEW`, `SUPPORTS`, `MANDATORY`, `NOT_SUPPORTED`, `NEVER`, `NESTED`.

---

## Lesson 4

## Spring ORM using Hibernate

---

### Hibernate at a Glance

- The most popular ORM; also a JPA provider
- Core objects:
  - **`SessionFactory`**: heavyweight, one per database, thread-safe
  - **`Session`**: lightweight unit of work, **not** thread-safe
  - **`Transaction`**: unit of atomic work
- Query languages: **HQL**, Criteria API, native SQL
- Features: caching (L1/L2), lazy loading, dirty checking, schema generation

```xml
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.4.x</version>
</dependency>
```

---

### Mapping an Entity

```java
import jakarta.persistence.*;

@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(name = "name", nullable = false, length = 100)
    private String name;

    private double salary;

    @Enumerated(EnumType.STRING)
    private Grade grade;

    protected Employee() {}              // required by Hibernate
    public Employee(String name, double salary) {
```

```
        this.name = name; this.salary = salary;
    }
    // getters and setters
}
```

---

## Relationships

```
@Entity
public class Department {
    @Id @GeneratedValue
    private Long id;
    private String name;

    @OneToMany(mappedBy = "department", cascade = CascadeType.ALL,
               orphanRemoval = true)
    private List<Employee> employees = new ArrayList<>();
}

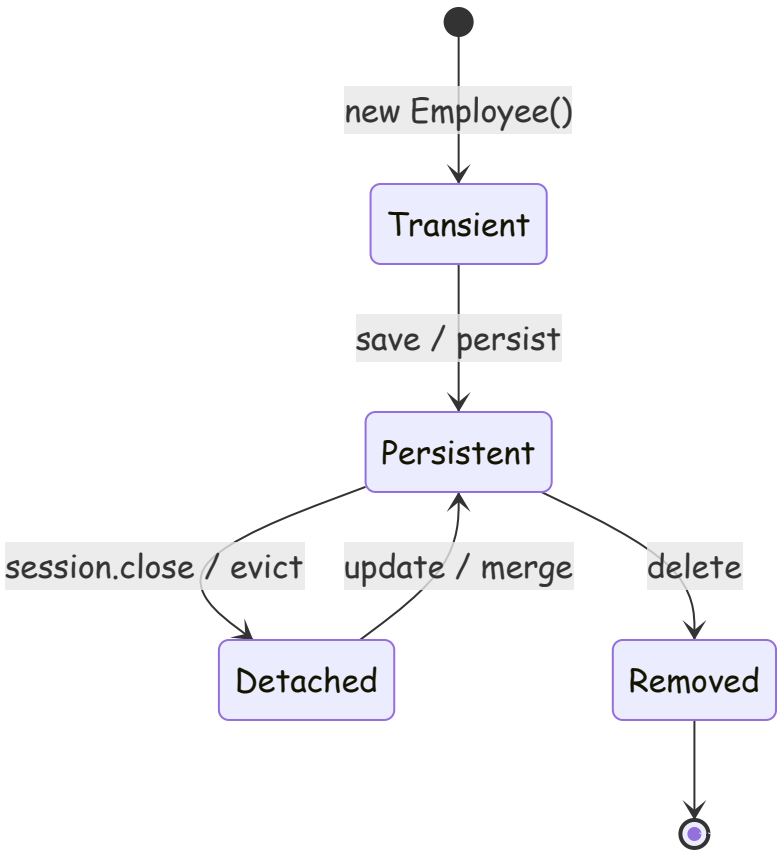
@Entity
public class Employee {
    @Id @GeneratedValue
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "department_id")
    private Department department;

    @ManyToMany
    @JoinTable(name = "employee_project",
        joinColumns = @JoinColumn(name = "employee_id"),
        inverseJoinColumns = @JoinColumn(name = "project_id"))
    private Set<Project> projects = new HashSet<>();
}
```

---

## Entity Lifecycle in Hibernate




- **Persistent** objects are tracked; changes are flushed automatically (**dirty checking**)
- **Detached** objects need `merge()` to be re-attached

---

### Native Hibernate API (no Spring)

```
SessionFactory sf = new Configuration().configure().buildSessionFactory();

try (Session session = sf.openSession()) {
    Transaction tx = session.beginTransaction();
    try {
        Employee e = new Employee("Omar", 6000);
        session.persist(e);
        tx.commit();
    } catch (RuntimeException ex) {
        tx.rollback();
        throw ex;
    }
}
```

`hibernate.cfg.xml`

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.url">jdbc:h2:mem:testdb</property>
    <property name="hibernate.dialect">org.hibernate.dialect.H2Dialect</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <mapping class="com.example.Employee"/>
  </session-factory>
</hibernate-configuration>
```

---

### Session CRUD Operations

```
// Create
session.persist(employee);

// Read
Employee e  = session.get(Employee.class, 1);     // null if missing
Employee p  = session.getReference(Employee.class, 1); // lazy proxy

// Update (managed entity: dirty checking, no explicit call)
e.setSalary(7000);

// Update (detached entity)
session.merge(detachedEmployee);

// Delete
session.remove(e);
```

---

## Lesson 5

## Spring ORM Hibernate Integration

---

### Integration Overview

1. Define `DataSource`
2. Define `LocalSessionFactoryBean`
3. Define `HibernateTransactionManager`
4. Enable `@Transactional`
5. Write `@Repository` using `sessionFactory.getCurrentSession()`
6. Write `@Service` with `@Transactional`

---

### Java Configuration

```
@Configuration
@EnableTransactionManagement
@ComponentScan("com.example")
public class HibernateConfig {

    @Bean
    public DataSource dataSource() {
        var ds = new DriverManagerDataSource();
        ds.setDriverClassName("org.h2.Driver");
        ds.setUrl("jdbc:h2:mem:company;DB_CLOSE_DELAY=-1");
        ds.setUsername("sa");
        return ds;
    }

    @Bean
```

```
    public LocalSessionFactoryBean sessionFactory(DataSource ds) {
        var sf = new LocalSessionFactoryBean();
        sf.setDataSource(ds);
        sf.setPackagesToScan("com.example.model");
        Properties p = new Properties();
        p.put("hibernate.dialect", "org.hibernate.dialect.H2Dialect");
        p.put("hibernate.hbm2ddl.auto", "create-drop");
        p.put("hibernate.show_sql", "true");
        p.put("hibernate.format_sql", "true");
        sf.setHibernateProperties(p);
        return sf;
    }

    @Bean
    public HibernateTransactionManager transactionManager(SessionFactory sf) {
        return new HibernateTransactionManager(sf);
    }

    @Bean
    public PersistenceExceptionTranslationPostProcessor translator() {
        return new PersistenceExceptionTranslationPostProcessor();
    }
}
```

---

### XML Configuration (Legacy Style)

```
<bean id="sessionFactory"
      class="org.springframework.orm.hibernate5.LocalSessionFactoryBean">
  <property name="dataSource" ref="dataSource"/>
  <property name="packagesToScan" value="com.example.model"/>
  <property name="hibernateProperties">
    <props>
      <prop key="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</prop>
      <prop key="hibernate.hbm2ddl.auto">update</prop>
    </props>
  </property>
</bean>

<bean id="transactionManager"
      class="org.springframework.orm.hibernate5.HibernateTransactionManager">
  <property name="sessionFactory" ref="sessionFactory"/>
</bean>

<tx:annotation-driven/>
```

Spring 6 uses `org.springframework.orm.hibernate5` classes on Hibernate 5.6 and the `SessionFactory` from Hibernate 6 through the same `LocalSessionFactoryBean` in `org.springframework.orm.jpa.hibernate`-style setups. Match the Spring and Hibernate versions to your BOM.

---

## The Repository (Hibernate DAO)

```
@Repository
public class EmployeeRepository {

    private final SessionFactory sessionFactory;

    public EmployeeRepository(SessionFactory sessionFactory) {
        this.sessionFactory = sessionFactory;
    }

    private Session session() {
        return sessionFactory.getCurrentSession();   // bound to Spring tx
    }

    public void save(Employee e)      { session().persist(e); }
    public Employee find(int id)      { return session().get(Employee.class, id); }
    public void delete(Employee e)    { session().remove(e); }

    public List<Employee> findAll() {
        return session().createQuery("from Employee", Employee.class).list();
    }

    public List<Employee> findByName(String name) {
        return session()
            .createQuery("from Employee where name like :n", Employee.class)
            .setParameter("n", "%" + name + "%")
            .list();
    }
}
```

`getCurrentSession()` only works **inside** a Spring-managed transaction.

---

## The Service Layer

```
@Service
public class EmployeeService {

    private final EmployeeRepository repo;
    public EmployeeService(EmployeeRepository repo) { this.repo = repo; }

    @Transactional
    public void hire(String name, double salary) {
        repo.save(new Employee(name, salary));
    }

    @Transactional(readOnly = true)
    public List<Employee> listAll() {
        return repo.findAll();
    }

    @Transactional
```

```
    public void raise(int id, double percent) {
        Employee e = repo.find(id);              // managed entity
        e.setSalary(e.getSalary() * (1 + percent / 100));
        // no save() needed: dirty checking flushes on commit
    }
}
```

---

### Running It

```
public class App {
    public static void main(String[] args) {
        try (var ctx = new AnnotationConfigApplicationContext(HibernateConfig.class)) {
            EmployeeService svc = ctx.getBean(EmployeeService.class);
            svc.hire("Nora", 5000);
            svc.hire("Khaled", 5500);
            svc.raise(1, 10);
            svc.listAll().forEach(e ->
                System.out.println(e.getId() + " " + e.getName() + " " + e.getSalary()));
        }
    }
}
```

Expected output (values may vary):

```
1 Nora 5500.0
2 Khaled 5500.0
```

---

## Lesson 6

### Spring ORM using JPA

---

### What is JPA?

- **Jakarta Persistence API**: a **specification**, not an implementation
- Vendor-neutral: Hibernate, EclipseLink, OpenJPA implement it
- Standard annotations ( `@Entity` , `@Id` ...), `EntityManager` , JPQL, Criteria API
- Package: `jakarta.persistence` (Jakarta EE 9+; earlier `javax.persistence` )

```
Your code → JPA API → Provider (Hibernate) → JDBC → Database
```

**Why JPA over native Hibernate?** Portability, one standard API, and the direction of the ecosystem (Spring Data JPA builds on it).

## JPA vs Native Hibernate API

| Concept | Hibernate | JPA |
|---|---|---|
| Factory | `SessionFactory` | `EntityManagerFactory` |
| Unit of work | `Session` | `EntityManager` |
| Save | `persist` / `save` | `persist` |
| Update detached | `update` / `merge` | `merge` |
| Query language | HQL | JPQL (subset of HQL) |
| Config file | `hibernate.cfg.xml` | `persistence.xml` |

Hibernate's `Session` **extends** `EntityManager` in Hibernate 6.

---

## JPA Entity

```
import jakarta.persistence.*;

@Entity
@Table(name = "product")
@NamedQuery(name = "Product.expensive",
            query = "select p from Product p where p.price > :min")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE)
    private Long id;

    @Column(nullable = false, unique = true)
    private String sku;

    private String name;

    @Column(precision = 10, scale = 2)
    private BigDecimal price;

    @Version                    // optimistic locking
    private Integer version;

    @Temporal(TemporalType.TIMESTAMP)
    private Date createdAt;

    // constructors, getters, setters
}
```

---

## persistence.xml (Standalone JPA)

`META-INF/persistence.xml`

```xml
<persistence xmlns="https://jakarta.ee/xml/ns/persistence" version="3.0">
  <persistence-unit name="companyPU" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <class>com.example.model.Product</class>
    <properties>
      <property name="jakarta.persistence.jdbc.url"
                value="jdbc:h2:mem:shop"/>
      <property name="jakarta.persistence.jdbc.user" value="sa"/>
      <property name="hibernate.hbm2ddl.auto" value="create-drop"/>
      <property name="hibernate.show_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

With Spring you can drop this file and configure everything in beans (Lesson 7).

---

## EntityManager API

```java
EntityManagerFactory emf = Persistence.createEntityManagerFactory("companyPU");
EntityManager em = emf.createEntityManager();

em.getTransaction().begin();

Product p = new Product();
p.setSku("A-100"); p.setName("Keyboard"); p.setPrice(new BigDecimal("49.90"));
em.persist(p);                          // INSERT on flush/commit

Product found = em.find(Product.class, p.getId());   // SELECT by PK
found.setPrice(new BigDecimal("44.90"));             // dirty checking => UPDATE

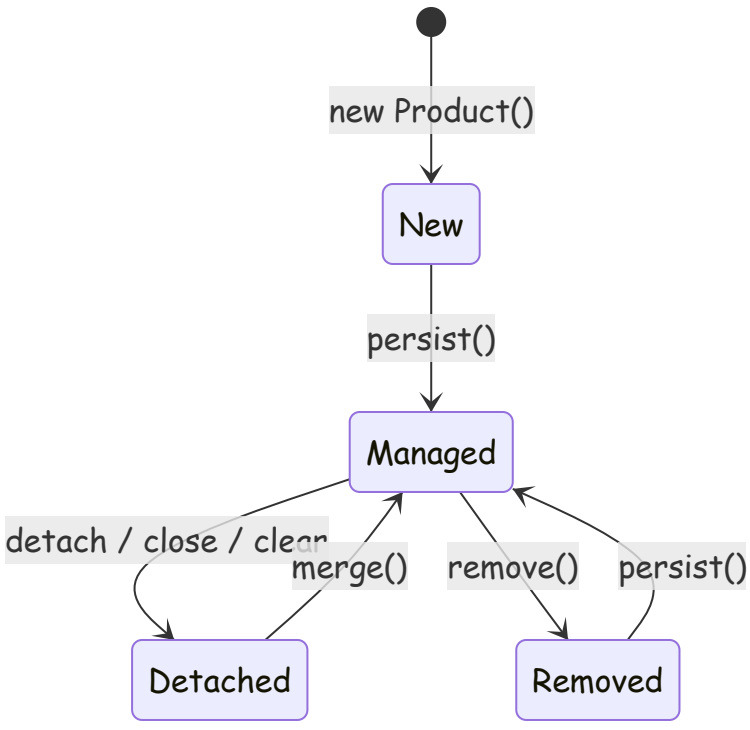
em.remove(found);                                    // DELETE

em.getTransaction().commit();
em.close();
```

Other operations: `merge`, `refresh`, `detach`, `flush`, `contains`, `getReference`.

---

## Persistence Context and Lifecycle




The **persistence context** is a first-level cache of managed entities. Within it, `find` on the same id returns the **same instance**, and changes are written at flush time.

---

## Lesson 7

## Spring ORM JPA Integration

---

### Integration Overview

1. Define `DataSource`
2. Define `LocalContainerEntityManagerFactoryBean` (with `HibernateJpaVendorAdapter`)
3. Define `JpaTransactionManager`
4. Enable `@Transactional`
5. Inject the `EntityManager` with `@PersistenceContext`
6. Write `@Repository` and `@Service`

---

### Java Configuration

```
@Configuration
@EnableTransactionManagement
@ComponentScan("com.example")
public class JpaConfig {

    @Bean
    public DataSource dataSource() {
        var ds = new DriverManagerDataSource();
        ds.setDriverClassName("org.h2.Driver");
        ds.setUrl("jdbc:h2:mem:shop;DB_CLOSE_DELAY=-1");
        ds.setUsername("sa");
        return ds;
    }
```

```
@Bean
public LocalContainerEntityManagerFactoryBean entityManagerFactory(DataSource ds) {
    var emf = new LocalContainerEntityManagerFactoryBean();
    emf.setDataSource(ds);
    emf.setPackagesToScan("com.example.model");   // no persistence.xml needed
    emf.setJpaVendorAdapter(new HibernateJpaVendorAdapter());

    Properties p = new Properties();
    p.put("hibernate.hbm2ddl.auto", "create-drop");
    p.put("hibernate.show_sql", "true");
    emf.setJpaProperties(p);
    return emf;
}

@Bean
public PlatformTransactionManager transactionManager(EntityManagerFactory emf) {
    return new JpaTransactionManager(emf);
}

@Bean
public PersistenceExceptionTranslationPostProcessor translator() {
    return new PersistenceExceptionTranslationPostProcessor();
}
}
```

---

### The JPA Repository

```
@Repository
public class ProductRepository {

    @PersistenceContext          // Spring injects a shared, thread-safe proxy
    private EntityManager em;

    public void save(Product p) {
        if (p.getId() == null) em.persist(p);
        else em.merge(p);
    }

    public Product find(Long id) {
        return em.find(Product.class, id);
    }

    public List<Product> findAll() {
        return em.createQuery("select p from Product p", Product.class)
                 .getResultList();
    }

    public Optional<Product> findBySku(String sku) {
        return em.createQuery("select p from Product p where p.sku = :sku",
                              Product.class)
                 .setParameter("sku", sku)
                 .getResultStream().findFirst();
    }
```

```
    public void delete(Long id) {
        Product p = em.find(Product.class, id);
        if (p != null) em.remove(p);
    }
}
```

The injected `EntityManager` is a proxy that delegates to the one bound to the **current transaction**.

---

### The Service and Main Class

```
@Service
public class ProductService {
    private final ProductRepository repo;
    public ProductService(ProductRepository repo) { this.repo = repo; }

    @Transactional
    public Product create(String sku, String name, BigDecimal price) {
        if (repo.findBySku(sku).isPresent())
            throw new IllegalStateException("Duplicate SKU " + sku);
        Product p = new Product(sku, name, price);
        repo.save(p);
        return p;
    }

    @Transactional
    public void discount(Long id, int percent) {
        Product p = repo.find(id);
        p.setPrice(p.getPrice()
            .multiply(BigDecimal.valueOf(100 - percent))
            .divide(BigDecimal.valueOf(100)));
    }

    @Transactional(readOnly = true)
    public List<Product> catalog() { return repo.findAll(); }
}
```

```
var ctx = new AnnotationConfigApplicationContext(JpaConfig.class);
var svc = ctx.getBean(ProductService.class);
svc.create("A-100", "Keyboard", new BigDecimal("49.90"));
svc.discount(1L, 10);
svc.catalog().forEach(p -> System.out.println(p.getName() + " " + p.getPrice()));
```

---

### Beyond Spring ORM: Spring Data JPA

Spring Data JPA removes the repository boilerplate:

```java
public interface ProductRepository extends JpaRepository<Product, Long> {
    Optional<Product> findBySku(String sku);
    List<Product> findByPriceLessThanOrderByNameAsc(BigDecimal max);

    @Query("select p from Product p where p.name like %:kw%")
    List<Product> search(@Param("kw") String keyword);
}
```

No implementation class is needed. Spring generates it at runtime. Add `@EnableJpaRepositories` (Spring Boot does this automatically).

---

### Comparison: Which Approach When?

| Approach | Strength | Trade-off |
|---|---|---|
| Spring JDBC (`JdbcTemplate`) | Full SQL control, fast, simple | Manual mapping, verbose for large domains |
| Spring + native Hibernate | Full Hibernate features | Tied to Hibernate API |
| Spring + JPA | Standard API, portable | Some Hibernate features need extensions |
| Spring Data JPA | Least code | Abstraction can hide queries; learn its conventions |

**Tip:** mixing is fine. Use JPA for the domain model and `JdbcTemplate` for reports and bulk operations, both in the same transaction manager (`JpaTransactionManager` supports it).

---

### Course Summary

1. **JDBC in Spring**: `DataSource` beans, pooling, no manual connection code
2. **Spring JDBC**: `JdbcTemplate`, `RowMapper`, named parameters, `DataAccessException`
3. **Spring ORM**: abstraction, exception translation, transaction managers
4. **Hibernate**: entities, `Session`, HQL, relationships, lifecycle
5. **Hibernate integration**: `LocalSessionFactoryBean` + `HibernateTransactionManager`
6. **JPA**: specification, `EntityManager`, JPQL, persistence context
7. **JPA integration**: `LocalContainerEntityManagerFactoryBean` + `JpaTransactionManager`

---

### Practice Exercises

- Build an `Employee` CRUD DAO with `JdbcTemplate` and H2; add a batch insert
- Convert it to Hibernate: `Department` 1-N `Employee`, and fix an N+1 with `join fetch`
- Convert again to JPA with `@PersistenceContext`
- Add a service method that throws midway and verify the **rollback**

- Bonus: replace your repository with Spring Data JPA and compare code size

**Questions?**